

stepik

Делаем свой AI-продукт на базе ChatGPT или других LLM моделей.

Прогресс по курсу: 124/201

3.1 Память в LangChain

3.2 Chains - собери сво...

3.3 Агенты intro

4 LLM и ваши данные | ...

4.1 RAG с вашими дан...

4.2 Собери свою ...

4.3 Prompt Engineering - 6...

5 Дообучение на своих да...

5.1 Open Source модел...

4.1 RAG с вашими данными

11 из 11 шагов пройдено

21 из 21 балла получен

Литературный RAG из pdf

Если в школе перед уроком литературы чтение даже краткого содержания произведения казалось тяжелым испытанием, то теперь, благодаря RAG и LLM, вы можете создать собственное ультракраткое содержание и задавать по нему вопросы.

Что делать? Создайте свой RAG по тексту краткого содержания романа «Капитанская дочка» и ответьте на вопросы из предоставленного датасета.

Что на выходе? Датасет с вопросами в столбце question и pdf файл с кратким содержанием романа.

Загрузка файлов с помощью кода

	question
0	Кого убили раздетой на крыльце?
1	Где Маша остановилась в Петербурге?
2	Почему Гринев и Савельичн сблизис с дороги?
3	Кто издевался над Машей и морил ее голодом?
4	Куда приговорили отправить Гринева в ссылку?
5	Сколько детей было в семье Петра Гринева?
6	Кто воспитывал Петра Гринева?
7	Сколько казаков попросил Гринев, чтобы очистит...
8	Как далеко находилась от города Белогорская кр...
9	Как звали дочку коменданта?
10	Что Петя подарил вожатому?
11	Какое напутствие дал Петру отец?
12	Кого Пугачев оставил комендантом Белогорской к...
13	Почему Швабрина перевели в крепость?
14	Кого просил пощадить Савельичн у Пугачева?

Что на выходе? csv файл, содержащий два столбца - question, answer.

Замечание: Ваше решение будет зачтено, если в нём будет минимум 8 правильных ответов. Корректность ответа проверяем по ключевым словам. Поэтому не беспокойтесь о формате ответа, но главное - длина каждого ответа не должна превышать 70 символов (иначе этот ответ не будет засчитан)

План решения, если не знаете как начать:

Напишите текст

Верно решил 41 учащийся

Из всех попыток 66% верных

Вы пбили порог в 8: ваша точность 11 из 15.

4.1_11_solution.csv (3 KB)

Следующий шаг

Решить снова

Ваши решения

Вы получили: 8 баллов из 8

11 6 Шар 11

Следующий шаг

3 Комментария

4 Решения

Самые популярные

Будьте вежливы и соблюдайте наши принципы сообщества. Поделитесь своим решением и посмотрите, как задачу решали другие.

Оставить решение

Комментарий закреплен

Пётр Михайлов 4 месяца назад

Ноутбук с бейлайном:

https://colab.research.google.com/drive/1mVCWMTRicXr1_vMY0kD0RVtATmhuiHDq

Верное решение #1178603182

Вы пбили порог в 8: ваша точность 12 из 15.

4.1_11_solution(1).csv (2 KB)

Изменен Пётр Михайлов 4 месяца назад

Ответить

VR Vladimir Rudenko 3 месяца назад

В общем то всё по файлу из курса.

Лишь не получилось использовать embeddings от HuggingFaceEmbeddings, хотя перевести в вектора тексты удалось, поэтому использовал course embeddings = OpenAIEmbeddings(course_api_key=course_api_key)

import pandas as pd

from tqdm import tqdm

from langchain.schema import StrOutputParser

from langchain_core.prompts import ChatPromptTemplate

from langchain_core.runnables import RunnablePassthrough

from langchain_community.document_loaders import DirectoryLoader, UnstructuredPDFLoader

from langchain.text_splitter import RecursiveCharacterTextSplitter

from langchain.vectorstores import FAISS

from utils import OpenAIEmbeddings, ChatOpenAI

llm = ChatOpenAI(temperature=0.0, course_api_key=course_api_key)

df = pd.read_csv('pushkin_questions.csv')

loader = DirectoryLoader('pdf', glob='**/*.pdf', loader_cls=UnstructuredPDFLoader)

documents = loader.load()

splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=100, length_function=len,

texts = [doc.page_content for doc in documents]

split_documents = splitter.create_documents(texts)

embeddings = OpenAIEmbeddings(course_api_key=course_api_key)

db = FAISS.from_documents(split_documents, embeddings)

db.save_local("puskin_db")

retriever = db.as_retriever(search_type="similarity", k=4, score_threshold=None,

template = """

Ответь на вопросы основываясь только на следующем контексте:

{context}

Если ты не уверен в ответе исходя из контекста, то напиши "не знаю".

Длина каждого ответа не должна превышать 70 символов. Нужны четкие, краткие и прямые ответы.

Question: {question}

prompt = ChatPromptTemplate.from_template(template)

def format_docs(docs):

return "\n\n".join([d.page_content for d in docs])

chain = (

{ "context": retriever | format_docs, "question": RunnablePassthrough() }

| prompt

| llm

| StrOutputParser()

answers = []

for text_input in tqdm(df['question']):

answer = chain.invoke(text_input)

answers.append(answer)

break

df['answer'] = answers

df.to_csv('4_1_11_solution.csv', index=False)

Верное решение #1204074183

Вы пбили порог в 8: ваша точность 13 из 15.

4.1_11_solution.csv (2 KB)

Свернуть

1 Ответить

AM Albert Murtazin 5 месяцев назад

Верное решение #1154117972

Вы пбили порог в 8: ваша точность 15 из 15.

4.1_semantic_text_splitter.csv (2 KB)

1 Ответить Скрыть ответы

MM Максим Марков 5 месяцев назад

@Albert_Murtazin, круто! можете поделиться промптом?)

1 Ответить

AM Albert Murtazin 5 месяцев назад

@Максим_Марков, конечно

template = """

Ответь на вопрос кратким ответом, руководствуясь контекстом ниже.

Если не уверен в ответе исходя из контекста, то напиши "не знаю".

Контекст:

{context}

Вопрос:

{question}

Ответ:

"""

Изменен Albert Murtazin 5 месяцев назад

2 Ответить

AM Albert Murtazin 5 месяцев назад

Но тут скорее упор делался на на промпт, а на другие вещи.

Например, на чистке текста: отформатировал переносы, удалил ненужные ссылки и т.д.

Разбиение на чанки отдал на откуп токенизеру "bert-base-uncased"

tokenizer = Tokenizer.from_pretrained("bert-base-uncased")

splitter = HuggingFaceTextSplitter(tokenizer, trim_chunks=True)

chunks = splitter.chunks(text, chunk_capacity=(100,500))

Из rag db доставал по 10 чанков, да и вообще поигрался с параметрами:

retriever = db.as_retriever(search_kargs={'k': 10}, # search_type="similarity_score_threshold", # search_kargs={'k': 20, "score_threshold": 0.1},)

Использовал локальную модель saiga_mistral_7B с температурой 0.1. При разной генерации ответа на последний вопрос выдаёт ответ: "Гринева", "Савельича" и весьма!!! редкий вариант "барского дятля".

Получается последний вопрос с помощью LLM так и не победил стабильностью, только регенерацией (хотя "Гринева" вроде тоже правильный ответ, но не засчитывается).

Дополнительно пробовал требовать более развёрнутые ответы для контроля LLM в промпте. И конечно тестить разные локальные модели: "mistral_70B", "mistral_instruct_v0.1-v0.2", etc. для поиска более подходящей по задаче.

Поэтому тут работа скорее не с промптом (только более сложные ответы для контроля), а сколько со всем пайплайном в совокупности.

Свернуть

Изменен Albert Murtazin 5 месяцев назад

2 Ответить

PO Павел Орлов 4 месяца назад

@Albert_Murtazin, если не секрет, как локально запускаете? В смысле с помощью каких фреймворков и т.д. Мне интересно, на 10G GPU получится что-то запустить локально?

1 Ответить

AP Aleksandr Pylaev 4 дня назад

from dotenv import load_dotenv

import os

from langchain_openai import ChatOpenAI

import pandas as pd

from tqdm import tqdm

load_dotenv(dotenv_path='.env.txt') # load env variables

api_key = os.getenv("OPENAI_API_KEY") # load api key

os.environ["OPENAI_API_KEY"] = api_key # set api key

llm = ChatOpenAI(temperature=0.0) # set llm

df = pd.read_csv('pushkin_questions.csv') # read csv file from folder with project

from langchain_community.document_loaders import PyPDFLoader

file_path = "The_Daughter_of_The_Commandant.pdf"

loader = PyPDFLoader(file_path)

pages = loader.load_and_split()

Объединение всех страниц в один текст

full_text = "\n\n".join(page.page_content for page in pages)

from langchain.text_splitter import RecursiveCharacterTextSplitter

import re

можно по другому еще разбить сплитеры

Кастомный сплиттер для разделения по главам

class ChapterTextSplitter(RecursiveCharacterTextSplitter):

def __init__(self, chunk_size=1000, chunk_overlap=100, separators=None):

super().__init__(chunk_size=chunk_size, chunk_overlap=chunk_overlap, separators=separators)

def split_text(self, text):

Разделение текста по главам с использованием регулярного выражения

chapters = re.split(r'(\n\n[А-Яа-я]+[.!?;:"]+)', text)

chunks = []

for i in range(1, len(chapters), 2):

chapter_title = chapters[i]

chapter_text = chapters[i + 1]

chunks.append(chapter_title + chapter_text)

return chunks

Объединение всех страниц в один текст

full_text = "\n\n".join(page.page_content for page in pages)

Инициализация и использование кастомного сплиттера

splitter = ChapterTextSplitter(chunk_size=1000, chunk_overlap=100)

chapters = splitter.split_text(full_text)

len(chapters)

from langchain_openai import OpenAIEmbeddings

from langchain.docstore.document import Document

from langchain.vectorstores import FAISS

Преобразуем главы в список объектов Document... нужно преобразовать в формат который принимает метод from_documents

documents = [Document(page_content=chapter) for chapter in chapters]

embeddings_api_model = OpenAIEmbeddings()

db = FAISS.from_documents(documents, embeddings_api_model)

db.save_local("Pushkin_local")

Задаём retriever

retriever = db.as_retriever(search_type='similarity', k=3, score_threshold=None)

from langchain.schema import StrOutputParser

from langchain_core.prompts import ChatPromptTemplate

from langchain_core.runnables import RunnablePassthrough

Создаём простой шаблон!!! ВАЖНО. промпт можно улучшить, даже нужно. чтобы он более конкретные ответы давал

template = """

Ответь четко, просто и без лишних слов и символа поставленный вопрос согласно контексту:

{context}

Question: {question}

"""

Создаём промпт из шаблона

prompt = ChatPromptTemplate.from_template(template)

Объявляем функцию, которая будет собирать строку из полученных документов

def format_docs(docs):

#print(type(docs[0].page_content))

return "\n\n".join(docs[0].page_content for d in docs)

str_to_llm = format_docs(retriever.invoke("Кого убили раздетой на крыльце?"))

Создаём цепочку

chain = (

{ "context": retriever | format_docs, "question": RunnablePassthrough() }

| prompt

| llm

| StrOutputParser()

answers = [] # Список, где будем хранить ответы модели

for text_input in tqdm(df['question']):

answer = chain.invoke(text_input)

answers.append(answer) # Добавляем ответ в список

df['answer'] = answers

df.to_csv('4_1_11_solution.csv', index=False)

Верное решение #1261709379

Вы пбили порог в 8: ваша точность 12 из 15.

4.1_11_solutionNEW.csv (2 KB)

Свернуть

1 Ответить